Manual Técnico

Sistema de Ventas e Inventario

Jeffrey Anderson Boror Ortiz

1. Descripción general

El Sistema de Ventas e Inventario es un programa de escritorio hecho en Java. Sirve para registrar productos, clientes, empleados, bodegas, el inventario de cada bodega y las órdenes de compra. Cuando una venta se completa, el programa resta del inventario los productos vendidos.

El programa usa Programación Orientada a Objetos, tiene una interfaz gráfica hecha con Swing y guarda la información en archivos de texto, así los datos no se pierden al cerrarlo.

2. Requisitos

Elemento	Detalle
Lenguaje	Java 17 o superior
Interfaz gráfica	Java Swing (ya viene con Java)
Librerías externas	Ninguna
Persistencia	Archivos `.txt` en la carpeta `datos/`
Control de versiones	Git y GitHub
IDE	NetBeans (el proyecto ya viene listo para abrirse)

3. Estructura del proyecto

```
SistemaVentasInventario/
├── src/
│   ├── Main.java              (inicio del programa)
│   ├── modelo/
│   │   ├── Persona.java       (clase padre)
│   │   ├── Cliente.java       (hereda de Persona)
│   │   ├── Empleado.java      (hereda de Persona)
│   │   ├── Producto.java
│   │   ├── DetalleProducto.java
│   │   ├── OrdenCompra.java
│   │   ├── Bodega.java
│   │   └── Inventario.java
│   ├── logica/
```

```
│   │   └── Tienda.java        (listas y reglas del sistema)
│   ├── persistencia/
│   │   └── ArchivoDatos.java  (leer y guardar archivos)
│   └── vista/
│       ├── Estilo.java         (colores y diseño)
│       ├── VentanaPrincipal.java
│       ├── PanelInicio.java
│       ├── PanelProductos.java
│       ├── PanelClientes.java
│       ├── PanelEmpleados.java
│       ├── PanelBodegas.java
│       ├── PanelInventario.java
│       └── PanelOrdenes.java
├── nbproject/                 (proyecto de NetBeans)
├── build.xml
├── manifest.mf
├── datos/                     (archivos .txt)
├── docs/                      (UML y manuales)
└── .gitignore
```

4. Cómo está organizado

El código está dividido en cuatro paquetes:

1. **modelo**: las clases que representan las cosas del negocio (producto, cliente, orden, etc.). Solo tienen atributos, constructor, getters, setters y algunos métodos sencillos.
2. **logica**: la clase `Tienda` guarda todas las listas (`ArrayList`) y tiene los métodos para buscar, agregar y actualizar el stock.
3. **persistencia**: la clase `ArchivoDatos` guarda las listas en archivos de texto y las vuelve a leer al abrir el programa.
4. **vista**: las ventanas. Cada pantalla es un panel que lee los datos del formulario, los valida con `if` y llama a la `Tienda`.

Cómo está hecha la interfaz

- **VentanaPrincipal** (hereda de `JFrame`) tiene un **menú lateral** con botones a la izquierda y el contenido a la derecha. El contenido usa `CardLayout`, que guarda todas las pantallas una encima de otra y muestra solo una a la vez. Cuando se presiona un botón del menú, un `if / else if` decide qué pantalla mostrar con `cartas.show(contenido, "productos")` y antes llama a `actualizar()` para recargar los datos.
- **PanelInicio** muestra un resumen: cuántos productos, clientes, bodegas y órdenes pendientes hay. Las pendientes se cuentan con un `for` y un `if`.
- **Los demás paneles** (heredan de `JPanel`) tienen la misma forma: un título arriba, el formulario a la izquierda en una tarjeta blanca y la tabla a la derecha. Cada panel implementa `ActionListener` y en `actionPerformed()` usa `if (e.getSource() == boton)` para saber qué botón se presionó.

- **Estilo** guarda los colores y tipos de letra, y tiene métodos que crean los botones, campos y tablas con el mismo diseño. Así todas las pantallas se ven iguales y, si se quiere cambiar un color, solo se cambia en esta clase.

5. Diagrama de clases (UML)

Diagrama de clases - Sistema de Ventas e Inventario

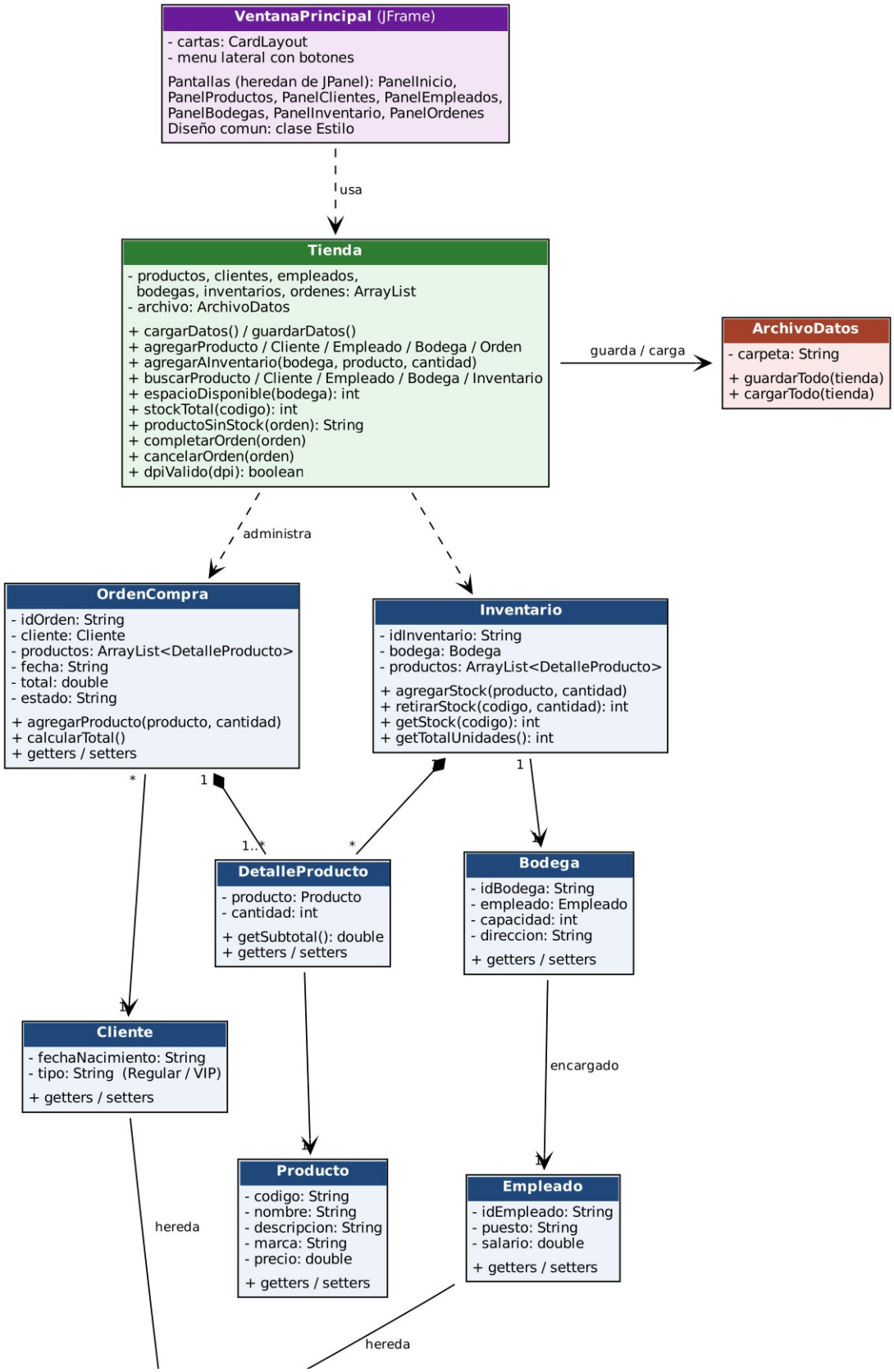


Diagrama de clases UML

Relaciones principales:

- **Herencia**: `Cliente` y `Empleado` heredan de `Persona` los atributos DPI, nombre y dirección. En su constructor llaman a `super(dpi, nombre, direccion)`.
- **Composición**: una `OrdenCompra` tiene una lista de `DetalleProducto` (producto y cantidad). Un `Inventario` también tiene una lista de `DetalleProducto`.
- **Asociación**: la orden tiene un `Cliente`, la bodega tiene un `Empleado` encargado, el inventario pertenece a una `Bodega` y cada `DetalleProducto` tiene un `Producto`.

6. Conceptos de POO aplicados

Encapsulamiento. Todos los atributos de las clases son `private`. Para leerlos o cambiarlos se usan métodos `get` y `set`. Ejemplo de la clase `Producto`:

```
private double precio;

public double getPrecio() {
    return precio;
}

public void setPrecio(double precio) {
    this.precio = precio;
}
```

Herencia. `Cliente` y `Empleado` heredan de `Persona` con `extends`, así no se repiten los atributos comunes:

```
public class Cliente extends Persona {
    private String fechaNacimiento;
    private String tipo;

    public Cliente(String dpi, String nombre, String direccion,
                   String fechaNacimiento, String tipo) {
        super(dpi, nombre, direccion);
        this.fechaNacimiento = fechaNacimiento;
        this.tipo = tipo;
    }
}
```

La interfaz también usa herencia: cada panel hereda de `JPanel` y la ventana hereda de `JFrame`.

Condicionales. Se usan `if / else if / else` para validar los datos antes de registrarlos, para saber qué botón se presionó y para revisar el estado de una orden. Ejemplo del panel de clientes:

```
if (dpi.isEmpty() || nombre.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Llene todos los campos.");
} else if (!tienda.dpiValido(dpi)) {
    JOptionPane.showMessageDialog(this, "El DPI debe tener exactamente
13 numeros.");
} else if (tienda.buscarCliente(dpi) != null) {
    JOptionPane.showMessageDialog(this, "Ya existe un cliente con ese
DPI.");
} else {
    tienda.agregarCliente(new Cliente(dpi, nombre, direccion, fecha,
tipo));
}
```

Listas y ciclos. Los registros se guardan en `ArrayList` y se recorren con ciclos `for` para buscar, sumar o llenar las tablas.

7. Persistencia en archivos

Los datos se guardan en la carpeta `datos/`. Cada archivo tiene un registro por línea y los campos se separan con punto y coma (`;`). En el inventario y las órdenes, la lista de productos se escribe como `codigo=cantidad`, separados con `/`.

```
productos.txt    codigo;nombre;descripcion;marca;precio
clientes.txt     dpi;nombre;direccion;fechaNacimiento;tipo
empleados.txt    idEmpleado;nombre;dpi;direccion;puesto;salario
bodegas.txt      idBodega;idEmpleado;capacidad;direccion
inventarios.txt  idInventario;idBodega;codigo=cantidad/codigo=cantidad
ordenes.txt
idOrden;dpiCliente;fecha;estado;total;codigo=cantidad/...
```

Ejemplo real de `ordenes.txt`:

```
OC-1;3012456780108;2026-11-03;Completada;78300.0;P001=12/P002=2
```

- **Guardar**: después de cada registro o cambio, `Tienda` llama a `guardarDatos()`, que reescribe todos los archivos con `PrintWriter` y `FileWriter`.
- **Cargar**: al abrir el programa, `ArchivoDatos` lee cada archivo línea por línea con `BufferedReader` y separa los campos con `split(";")`.
- **Orden de carga**: primero productos, clientes y empleados; después bodegas (necesitan al empleado), inventarios (necesitan bodega y productos) y por último órdenes (necesitan cliente y productos).
- Si el usuario escribe `;` en un texto, se cambia por `,` para no dañar el archivo.

8. Validaciones

Registro	Validaciones
Producto	Campos obligatorios, código no repetido,

Registro	Validaciones
	precio numérico mayor que cero.
Cliente	Campos obligatorios, DPI de 13 números y no repetido, fecha con formato dd/mm/aaaa.
Empleado	Campos obligatorios, ID no repetido, DPI de 13 números, salario mayor que cero.
Bodega	Debe existir un empleado, ID no repetido, capacidad entera mayor que cero.
Inventario	Cantidad entera mayor que cero y que no pase el espacio disponible de la bodega.
Orden	Debe tener cliente y al menos un producto.

9. Órdenes y actualización del stock

1. Al registrar una orden queda en estado **Pendiente**, con la fecha del día y un número (`OC-1`, `OC-2`, …). El total se calcula sumando precio × cantidad de cada producto.
2. Al presionar **Completar venta**, el panel revisa con `if` que la orden esté Pendiente.
3. Luego llama a `tienda.productoSinStock(orden)`, que revisa que haya stock suficiente de **todos** los productos. Si falta alguno, se muestra el nombre del producto y no se cambia nada.
4. Si hay stock, `tienda.completarOrden(orden)` resta las cantidades del inventario. Empieza por la primera bodega y, si no alcanza, sigue con la siguiente.
5. La orden pasa a **Completada** y se guardan los archivos.

Cancelar orden solo cambia el estado a **Cancelada**. El inventario no cambia porque todavía no se había descontado nada.

10. Compilar y ejecutar

En la carpeta del proyecto:

```
javac -d out src/Main.java src/modelo/*.java src/logica/*.java
src/persistencia/*.java src/vista/*.java
java -cp out Main
```

O con el JAR incluido: `java -jar SistemaVentasInventario.jar`

En NetBeans: el proyecto ya incluye sus archivos de NetBeans (carpeta `nbproject`, `build.xml` y `manifest.mf`). Se abre con **File → Open Project** seleccionando la carpeta `SistemaVentasInventario`. Con **Run** se ejecuta la clase `Main`, y con **Clean and Build** se genera el ejecutable en `dist/SistemaVentasInventario.jar`.

11. Pruebas realizadas

Se probó con datos de ejemplo que:

- No se aceptan campos vacíos, códigos o DPI repetidos, DPI incorrectos ni precios en cero.
- No se puede pasar la capacidad de una bodega.
- No se puede registrar una orden sin productos.
- Al completar una venta el stock baja exactamente lo vendido, aunque el producto esté en varias bodegas.
- Si falta stock la venta no se completa y el inventario no cambia.
- Al cerrar y abrir el programa los datos se cargan igual desde los archivos.

12. Control de versiones

El código se maneja con Git y se publica en GitHub. El archivo `.gitignore` evita subir los archivos compilados (`out/`, `build/`, `dist/`, `*.class`). No se suben las carpetas `build/` y `dist/` que crea NetBeans al compilar.